linalg_pedagopy — Plan de développement v5 (révisé)

Corriger et publier, ajouter le déterminant, consolider l'architecture, puis valeurs propres et diagonalisation

Cédric Bohnert

20 octobre 2026

Table des matières

1. Objet et résumé **2**
1.1 Correspondance avec la première version du plan . . . 3

2. État des lieux (20 octobre 2026) **3**
2.1 Historique récent de la branche `development` . . . 3
2.2 Mesures . . . 3
2.3 Bilan de la phase 0 . . . 4
2.4 Constats techniques . . . 4

3. Principes de conduite **4**

4. Phase 0 — Filet de sécurité (terminée) **5**

5. Phase 1 — Corrections et préparation de la publication **5**
5.1 Corrections (un commit chacune, références régénérées et relues) . . . 5
5.2 Préparation de la publication . . . 5
5.3 Critères de fin . . . 6

6. Phase 2 — Déterminant exact **6**
6.1 Le piège de « l'élimination sans dilatation » . . . 6
6.2 Algorithme de Bareiss . . . 6
6.3 Fonctionnalités . . . 7
6.4 Rendu et locales . . . 7
6.5 Intégration . . . 7
6.6 Tests . . . 8
6.7 Critères de fin . . . 8

7. Phase 3 — Moteur d'élimination réutilisable **8**
7.1 Responsabilités actuellement mélangées dans `_solve_under_condition` . . . 8
7.2 Conception proposée . . . 8
7.3 Étapes . . . 9
7.4 Risques . . . 9

8. Phase 4 — Alléger `Matrix` **9**

9. Phase 5 — Découpage du tuteur, suppression des `eval`, API française **9**
9.1 Arborescence cible . . . 10
9.2 Tâches . . . 10

10. Phase 6 — Valeurs propres, espaces propres, diagonalisation **11**
10.1 Fonctionnalités . . . 11

10.2 Choix mathématiques . . . 11
10.3 Pédagogie . . . 11
10.4 Tests . . . 11

11. Phase 7 — Extensions ultérieures **12**

12. Calendrier et dépendances **12**

13. Risques et parades **13**

14. Décisions **13**
14.1 Décisions prises (20 octobre 2026) . . . 13
14.2 Questions encore ouvertes . . . 13

15. Définition de « terminé » (pour chaque étape) **14**

Annexe A — Point de reprise pour la prochaine séance **14**

Annexe B — Esquisse de l'API cible (version 3.2) **14**

1. Objet et résumé

Ce plan organise la suite du développement de `linalg_pedagopy` sur la branche `development`. La première version (19 octobre 2026) plaçait toute la consolidation de l'architecture avant les nouvelles fonctionnalités. Cette révision (20 octobre 2026) tient compte de la phase 0 achevée et de trois décisions :

- **Le déterminant passe avant le moteur d'élimination.** Le moteur sera extrait quand il aura deux utilisateurs concrets (Gauss-Jordan et le déterminant), plutôt que conçu pour un seul.
- **Le dépôt sera public (GitHub et PyPI).** Une phase de corrections et de préparation de la publication est ajoutée juste après la phase 0.
- **Le français devient l'API principale du tuteur** ; les noms anglais restent des alias.

Le plan comporte **huit phases**, chacune livrée en commits indépendants, avec la suite de tests verte avant et après :

Phase	Contenu	Nature	Effort estimé	Version
0	Filet de sécurité : couverture, tests de caractérisation, intégration continue	outillage	**fait**	—
1	Corrections relevées en phase 0 et préparation de la publication	correctifs + hygiène	1 à 2 séances	3.0.1
2	**Déterminant** exact (Bareiss, élimination, cofacteurs, Sarrus)	fonctionnalité	2 à 3 séances	3.1.0
3	Extraction d'un **moteur d'élimination** (clients : Gauss-Jordan et déterminant)	refactorisation	2 à 3 séances	3.1.1
4	Allègement de `Matrix` (règle de délégation)	refactorisation	1 séance	3.1.1
5	Découpage du tuteur, suppression des `eval`, API française principale	refactorisation + sécurité	2 séances	3.1.1
6	**Valeurs propres, espaces propres, diagonalisation**	fonctionnalité	3 à 4 séances	3.2.0
7	Extensions ultérieures (exercices paramétrés, sous-espaces, géométrie, LU…)	feuille de route	—	—

Une *séance* désigne une demi-journée à une journée de travail. Les phases 3 à 5 ne changent **aucun comportement visible** : les sorties des carnets et de la documentation doivent rester identiques, et les tests de caractérisation le garan-

tissent. La phase 1 change délibérément quelques sorties (corrections), chacune par un commit dédié avec régénération et relecture des références.

1.1 Correspondance avec la première version du plan

Première version	Version révisée
Phase 0 — Filet de sécurité	Phase 0 (inchangée, terminée)
—	**Phase 1 — Corrections et publication** (nouvelle)
Phase 4 — Déterminant	Phase 2
Phase 1 — Moteur d'élimination	Phase 3
Phase 2 — Alléger `Matrix`	Phase 4
Phase 3 — Découpage du tuteur	Phase 5
Phase 5 — Valeurs propres	Phase 6
Phase 6 — Extensions	Phase 7

2. État des lieux (20 octobre 2026)

2.1 Historique récent de la branche `development`

Commit	Contenu
`9b81aa0`	Couverture des tests, guide des vérifications, CHANGELOG de la phase 0
`b7d029b`	Exécution des carnets, `verifier_tout.py`, workflow GitHub Actions
`536f01b`	Tests de caractérisation : traces, oracle et 35 fiches figés en références
`caa3c05`	Python 3.10 minimum, configuration ruff et mypy, dépendances de développement
`29441ca`	Corrections ruff sans effet sur le comportement

Non commité : cinq carnets modifiés (sorties ré-exécutées), `essai3` et `essai4` non suivis.

2.2 Mesures

- **Code** : environ 11 350 lignes dans `src/linalg_pedagopy`.
- **Tests** : **145 tests** (71 tests unitaires et 74 tests de caractérisation, dont un par fiche), tous verts.
- **Couverture** : 84,3 % au total (mesure approchée, voir `docs/dev/couverture.md`) ; `gauss_jordan.py` 94,7 %, `tutor/verifiers.py` 81,7 %, `core/zero.py` 68,2 %, `tutor/lock.py` 66 %, `tutor/panel.py` 24 %, `plot/geometry.py` 0 %.
- **Documentation** : 9 guides Markdown, site HTML généré (`build_docs_site.py`), tutoriel, carnets, guide développeur (`docs/dev/`).

Les plus gros modules :

Module	Lignes	Observation
`tutor/verifiers.py`	1 071	9 vérificateurs et leurs utilitaires dans un seul fichier
`tutor/catalog.py`	918	catalogue, séance, rapport de validation et contrôle croisé mélangés
`objects/matrix.py`	777	délègue déjà `rref`, `rank`, `nullspace`, `inv`, `solve` à `GaussJordan`
`exercises/generators.py`	722	20 générateurs d'exercices jumeaux
`algorithms/gauss_jordan.py`	699	**`_solve_under_condition` fait à elle seule environ 290 lignes**

2.3 Bilan de la phase 0

Les critères de fin sont atteints, **sauf la CI verte** : le dépôt n'a pas encore de remote, et la branche principale locale s'appelle `master` alors que le workflow se déclenche sur `main` et `development`. Le workflow n'a donc jamais tourné. Ce critère passe en phase 1 (tâche 1.9).

La caractérisation a relevé quatre comportements discutables, figés tels quels dans les références (`docs/dev/couverture.md`) :

1. `explain(level=0)` renvoie un texte vide, alors que la documentation présente le niveau 0 comme « résultat seul ».
2. Pour `a*b`, l'oracle renvoie bien les deux facteurs, mais sa `condition` ne mentionne que `a = 0`.
3. Pour `(a**2 - 1)/(a - 1)`, l'oracle simplifie d'abord et ne retient que le facteur `a + 1` : **la valeur interdite `a = 1` est perdue**. C'est une erreur mathématique (un résultat présenté comme valable partout ne l'est pas pour `a = 1`), qui toucherait directement le déterminant et les valeurs propres.
4. Une réponse illisible du tuteur recopie le message brut de SymPy et de Python, en anglais et variable selon la version.

S'y ajoutent environ 215 erreurs `mypy --strict`, dont 161 `attr-defined` dues au motif `__slots__` + `object.__setattr__`.

2.4 Constats techniques

1. **La logique d'élimination n'est pas réutilisable.** Le choix du pivot, l'interrogation de l'oracle, la séparation en cas (`_split_on_factors`) et l'enregistrement de la trace sont imbriqués dans `GaussJordan._solve_under_condition`.
2. **`Matrix` grossit par ajout de méthodes.** Ajouter `det()`, `eigenvalues()`, `diagonalize()` directement ferait dépasser 1 000 lignes.
3. **Le tuteur concentre deux gros fichiers** qui évoluent pour des raisons différentes.
4. **Douze appels à `eval`** subsistent (`tutor/lab.py` ×7, `verifiers.py` ×2, `catalog.py`, `hints.py`, `panel.py`), tous avec `__builtins__` vide ; un interpréteur fondé sur `ast` serait plus sûr et plus lisible.
5. **Le dépôt n'est pas prêt pour la publication** : un manuel PDF de 28 Mo (« Algèbre linéaire et géométrie vectorielle.pdf »), vraisemblablement protégé par le droit d'auteur, est suivi par git ; la racine mélange scripts, carnets d'essai et anciens plans ; les sorties des carnets créent du bruit dans chaque diff ; l'origine des fiches (recueil VMLS de Boyd et Vandenberghe, mentionné dans le CHANGELOG 3.0.0) n'est plus attribuée dans le paquet.
6. **API dédoublée** : `tutor` / `tuteur`, `verify_*` / `verifier_*` sont deux surfaces publiques à maintenir.
7. **Manques mathématiques** : ni déterminant, ni polynôme caractéristique, ni valeurs propres, ni LU, ni sous-espaces comme objets.

3. Principes de conduite

- **Une étape = un commit** (ou une petite série), message en français au format `type(portée): résumé` (`refactor`, `feat`, `fix`, `test`, `docs`, `ci`, `chore`).
- **Refactorisation pure dans les phases 3 à 5** : aucun changement de sortie, garanti par les tests de caractérisation.
- **Toute correction de comportement** a son propre commit, régénère les références concernées (`LP_MAJ_REFERENCES=1`) et s'accompagne d'une relecture du diff des références.
- **Compatibilité de l'API publique** : tout nom importable aujourd'hui depuis `linalg_pedagopy`, `linalg_pedagopy.tutor` ou `linalg_pedagopy.tuteur` le reste au moins une version. Le français est l'API documentée du tuteur ; les noms anglais sont des alias non documentés.
- **Chaque fonctionnalité nouvelle arrive complète** : code, tests, entrées de locale `fr.yaml`, rendu (texte, LaTeX, Markdown, HTML), documentation (guide + référence API), au moins un exercice ou une fiche.
- **Le calcul reste exact et sur** $\mathbb{R}$; les cas indécidables passent par l'oracle et `CaseSplit`, jamais par un choix arbitraire. **Une trace affichée doit être valable pour toutes les valeurs des paramètres couvertes par sa branche.**
- **Extraire une abstraction quand elle a au moins deux utilisateurs concrets** (d'où l'ordre déterminant → moteur).

4. Phase 0 — Filet de sécurité (terminée)

Réalisée du 19 au 20 octobre 2026 : couverture mesurée (`docs/dev/couverture.md`), tests de caractérisation des traces, de l'oracle et des 35 fiches (`tests/references/`), exécution des carnets (`tests/run_notebooks.py`), `verifier_tout.py`, workflow GitHub Actions (Python 3.10 à 3.13, ruff, mypy informatif), `requires-python >= 3.10`. Seul le critère « CI verte sur un vrai serveur » reste ouvert (tâche 1.9).

5. Phase 1 — Corrections et préparation de la publication

Objectif : corriger les comportements relevés en phase 0 avant de bâtir dessus, et rendre le dépôt publiable.

5.1 Corrections (un commit chacune, références régénérées et relues)

N°	Tâche	Détail
1.1	Valeurs interdites de l'oracle	Relever les zéros des dénominateurs **avant** toute simplification, et les inscrire dans la condition de domaine : pour `(a**2 - 1)/(a - 1)`, l'oracle doit retenir `a ≠ 1`. Tests dédiés (fractions rationnelles en un et deux paramètres). Prérequis du déterminant.
1.2	Condition d'un produit	Pour `a*b`, la condition doit être `a = 0 ou b = 0`. Vérifier les conséquences sur les discussions de cas de Gauss-Jordan (références à relire).
1.3	`explain(level=0)`	Afficher le résultat seul, comme l'annonce la documentation.
1.4	Réponses illisibles du tuteur	Remplacer le message brut de SymPy et de Python par un message français propre au tuteur (avec la forme attendue) ; le texte devient indépendant de la version de Python.
1.5	mypy	Annoter les attributs des classes immuables au niveau de la classe (motif `__slots__`), pour supprimer la plupart des 161 `attr-defined` ; viser `mypy --strict` sans erreur sur `core/` et `algorithms/`, puis rendre l'étape mypy **bloquante** sur ces dossiers.

5.2 Préparation de la publication

N°	Tâche	Détail
1.6	Retirer le manuel PDF	Retirer « Algèbre linéaire et géométrie vectorielle.pdf » **de l'historique** (`git filter-repo --path … --invert-paths`), avant le premier `push` : la réécriture change les empreintes des commits, ce qui est sans conséquence tant que rien n'est publié. Ajouter une règle `.gitignore` pour les ressources de référence locales.
1.7	Attribution des sources	Vérifier les conditions d'utilisation du recueil *Additional Exercises for Vectors, Matrices, and Least Squares* (Boyd et Vandenberghe) ; ajouter une section « Sources et remerciements » au `README.md` et à la documentation ; aligner le CHANGELOG 3.0.0 sur l'état actuel des fiches (identifiants lisibles, énoncés reformulés).

N°	Tâche	Détail
1.8	Ranger la racine	`scripts/` (`build_docs_site.py`, `build_notebook.py`, `build_project.py`, `convert_tutorial_to_ipynb.py`, `generate_exploratory_problems_ipynb.py`, `demo.py`), `carnets/` (tutoriel, problèmes, projet), `docs/dev/plans/` (plan v4 et plan v5). Les carnets `essai1` à `essai4` sortent du dépôt (ou vont dans `archives/` s'ils doivent être conservés). Mettre à jour les chemins dans `verifier_tout.py`, `tests/run_notebooks.py`, le workflow et la documentation.
1.9	Remote et CI	Créer le dépôt GitHub public, renommer `master` en `main`, pousser `main` et `development`, et obtenir un premier **pipeline vert** (critère de fin hérité de la phase 0).
1.10	Sorties des carnets	`nbstripout` (ou filtre `.gitattributes`) sur les carnets suivis, pour que les diffs ne montrent que le contenu ; la CI les exécute de toute façon.
1.11	Métadonnées du paquet	Classificateur `Development Status :: 4 - Beta` tant que l'API du tuteur évolue (phase 5) ; URL du dépôt et de la documentation dans `pyproject.toml`.

5.3 Critères de fin

- Les quatre comportements de la phase 0 sont corrigés, références régénérées et relues.
- Le dépôt public ne contient plus le manuel PDF dans son historique ; les sources sont attribuées.
- Pipeline CI vert sur GitHub, mypy bloquant sur `core/` et `algorithms/`.
- Version **3.0.1** étiquetée.

6. Phase 2 — Déterminant exact

Objectif : livrer le déterminant de façon autonome, en réutilisant les fonctions pures de `gauss_jordan.py` (`_simplify_entry`, `_pivot_complexity_key`) et l'oracle, sans attendre le moteur. Le code d'élimination écrit ici servira de second cas d'usage pour concevoir le moteur en phase 3.

6.1 Le piège de « l'élimination sans dilatation »

Une transvection $L_i \leftarrow L_i - \frac{a_{ik}}{p} L_k$ divise par le pivot p. Avec des paramètres, chaque étape n'est valable que si $p \neq 0$: le résultat final, une fois simplifié, reste juste pour toutes les valeurs (c'est une identité polynomiale), mais **la trace affichée est fausse** pour les valeurs qui annulent un pivot intermédiaire. D'où le choix de l'algorithme de Bareiss quand la matrice contient des paramètres.

6.2 Algorithme de Bareiss

Élimination sans fractions, avec $m_{00}^{(0)} = 1$:

$$m_{ij}^{(k)} = \frac{m_{kk}^{(k-1)}\, m_{ij}^{(k-1)} - m_{ik}^{(k-1)}\, m_{kj}^{(k-1)}}{m_{k-1,k-1}^{(k-2)}}$$

- La division est **exacte** dans l'anneau des polynômes : les entrées restent polynomiales en les paramètres, sans aucune discussion de cas.
- Le dernier pivot est le déterminant (au signe des échanges près).
- Un pivot ne doit être évité que s'il est **identiquement nul** (en tant que polynôme) : on échange alors avec une ligne dont le coefficient n'est pas identiquement nul. L'oracle est interrogé en mode « nullité identique », jamais en mode conditionnel.

- Pédagogie : chaque entrée est un mineur 2×2 divisé par le pivot précédent ; la trace l'explique au niveau 2 et donne le mineur au niveau 3.

6.3 Fonctionnalités

Élément	Description
`Determinant(A, *, method="auto", assumptions=None)`	Procédure tracée dans `algorithms/determinant.py`. `method="auto"` : Bareiss si la matrice contient des paramètres, élimination sinon.
`method="bareiss"`	Algorithme du § 6.2 ; valable sans hypothèse sur les paramètres.
`method="elimination"`	Triangulation classique sans dilatation, $\det A = (-1)^{\text{échanges}} \prod$ pivots. Avec des paramètres, si un pivot n'est pas inconditionnellement non nul, la procédure s'arrête avec une explication et propose `method="bareiss"` ; la version avec `CaseSplit` arrivera avec le moteur (phase 3).
`method="cofactors"`	Développement suivant une ligne ou une colonne (pédagogique, limité à $n \leq 4$), choix automatique de la ligne ou colonne la plus creuse.
`method="sarrus"`	Pour $n = 3$ uniquement.
`DetResult`	`value` (expression factorisée), `factors`, `method`, `trace` ; `explain()`, `verify()` (comparaison avec une autre méthode).
`zeros()`	Valeurs des paramètres annulant le déterminant, via la factorisation, l'oracle et `Condition` (`a = 1`, `a = -2`). Option `split=True` : `CaseSplit` « inversible / non inversible ».
`Matrix.det(**kw)`	Méthode de commodité ; renvoie une expression (éventuellement paramétrée).
Propriétés tracées	Effet des opérations élémentaires (échange : signe, dilatation : facteur, transvection : invariance), $\det(AB) = \det A \det B$, $\det A^T = \det A$, matrices triangulaires et par blocs.

6.4 Rendu et locales

Nouvelles clés `det.*` dans `render/locales/fr.yaml` (début, pivot, échange de signe, étape de Bareiss et mineur associé, produit final, développement par cofacteurs, mineur, arrêt de l'élimination sur un pivot conditionnel). Rendu LaTeX du développement : $\det A = \sum_j (-1)^{i+j} a_{ij} M_{ij}$ avec les mineurs affichés.

6.5 Intégration

- `Matrix.inv()` : en cas de paramètres, s'appuyer sur `det().zeros()` pour annoncer les cas singuliers avant l'élimination (message plus clair). Ce changement de message passe par la régénération des références concernées.
- Tuteur : 4 nouvelles fiches (thème « Déterminants ») : déterminant 3×3 par Sarrus, déterminant d'une matrice triangulaire par blocs, valeurs d'un paramètre rendant une matrice singulière, effet des opérations élémentaires. Vérificateurs existants (`valeur`, `propriete`) ; ajout éventuel d'un type `ensemble_valeurs`.
- Cahier d'exercices : remplacer l'indice « le déterminant vaut… » de l'exercice 16 par un appel possible à `det()` dans le corrigé.
- Documentation : section dans le guide 03, entrée dans la référence API, exemples dans le tutoriel.

6.6 Tests

- Comparaison avec `sympy.Matrix.det()` sur 50 matrices aléatoires entières et rationnelles ($n \leq 5$) et sur des matrices paramétrées (tests de propriétés avec `hypothesis`, déjà dans les dépendances).
- Égalité des quatre méthodes sur $n = 3$; égalité Bareiss / élimination / cofacteurs sur $n = 4$.
- Bareiss : entrées intermédiaires polynomiales (aucun dénominateur) sur des matrices paramétrées.
- Propriétés : multiplicativité, transposée, signe d'un échange.
- Cas limites : matrice nulle, 1×1, pivot identiquement nul imposant un échange, non carrée (`DimensionMismatchError`).

6.7 Critères de fin

`det()` disponible avec ses quatre méthodes, 4 fiches validées, documentation à jour — version **3.1.0**.

7. Phase 3 — Moteur d'élimination réutilisable

Objectif : isoler la machinerie commune à Gauss-Jordan et au déterminant dans `algorithms/elimination.py`, puis reconstruire les deux procédures dessus **sans changer leurs sorties**. Le moteur est conçu à partir de ces deux utilisateurs réels et des besoins connus de la phase 6 (noyaux $\ker(A - \lambda I)$).

7.1 Responsabilités actuellement mélangées dans `_solve_under_condition`

1. Construction de la matrice augmentée et simplification des coefficients.
2. Parcours colonne par colonne, recherche des candidats pivots et interrogation de l'oracle.
3. Bascule vers une discussion de cas quand aucun candidat n'est inconditionnellement non nul.
4. Choix du pivot selon la stratégie (`simplest`, `first`, `partial`).
5. Opérations élémentaires (échange, dilatation, transvection) et leur enregistrement dans la trace.
6. Test de compatibilité des lignes nulles, construction de l'inverse et de l'ensemble des solutions.

Les points 1 à 5 sont génériques ; le point 6 est propre à Gauss-Jordan.

7.2 Conception proposée

```python
@dataclass
class EliminationState:
    grid: list[list[sp.Expr]]       # matrice de travail (éventuellement augmentée)
    condition: Condition            # hypothèses de la branche courante
    pivots: list[tuple[int, int]]   # (ligne, colonne) des pivots trouvés
    swaps: int                      # nombre d'échanges (signe du déterminant)
    scale: sp.Expr                  # produit des facteurs de dilatation appliqués

class EliminationEngine:
    def __init__(self, *, pivot_strategy, reduce_above: bool, normalize_pivots: bool,
                 fraction_free: bool, split_col: int | None,
                 indeterminates: frozenset[sp.Symbol], max_depth: int, max_cases: int): ...
    def run(self, A_aug: Matrix, cond: Condition, depth: int,
            on_complete: Callable[[EliminationState], R]) -> R | CaseSplit[R]: ...
```

- `reduce_above=True, normalize_pivots=True` : forme échelonnée réduite (Gauss-Jordan).
- `reduce_above=False, normalize_pivots=False` : triangulation sans dilatation (déterminant par élimination), qui obtient ici sa version avec `CaseSplit` pour les pivots conditionnels.
- `fraction_free=True` : étapes de Bareiss au lieu des transvections ; le moteur n'interroge l'oracle que sur la nullité identique.
- `indeterminates` : symboles **jamais** discutés par l'oracle (le λ du polynôme caractéristique en phase 6) ; un pivot comme $2 - \lambda$ est traité comme non nul en tant que polynôme.
- `on_complete` reçoit l'état final d'une branche et fabrique le résultat métier (`GJResult`, `DetResult`, …). La séparation en cas (`_split_on_factors`) devient une méthode du moteur, qui rappelle `run` sur chaque branche.

- Les clés de trace (`gj.pivot_chosen`, `gj.row_add`, `det.*`…) restent identiques ; le moteur reçoit un préfixe (`gj`, `det`).

7.3 Étapes

N°	Étape	Contrôle
3.1	Extraire les fonctions pures (`_simplify_entry`, `_pivot_complexity_key`, choix du pivot) dans `elimination.py`	tests verts
3.2	Extraire les opérations élémentaires tracées (`swap`, `scale`, `add_multiple`, étape de Bareiss)	traces identiques
3.3	Extraire la boucle d'élimination et `_split_on_factors` dans `EliminationEngine`	traces et `CaseSplit` identiques
3.4	Réécrire `GaussJordan._solve_under_condition` comme appel au moteur + `on_complete`	`gauss_jordan.py` < 400 lignes
3.5	Reconstruire `Determinant` (élimination et Bareiss) sur le moteur ; ajouter le `CaseSplit` du mode élimination paramétré	traces du déterminant identiques ; nouveaux tests pour le `CaseSplit`
3.6	Tests unitaires propres au moteur (mode triangulaire, comptage des échanges, indéterminées non discutées)	nouveaux tests verts

7.4 Risques

- **Régression subtile dans l'ordre des branches ou des étapes** : couverte par les tests de caractérisation.
- **Performance** : l'indirection coûte peu devant SymPy ; mesurer sur un 5×5 paramétré avant et après.

8. Phase 4 — Alléger `Matrix`

Objectif : fixer une règle simple avant d'ajouter `eigenvalues()` et `diagonalize()`.

Règle de délégation : `Matrix` contient la représentation, les opérations algébriques (`+`, `@`, transposée, blocs, accès) et des **méthodes de commodité d'une à trois lignes** qui délèguent à une procédure de `algorithms/`. Toute logique d'algorithme (boucle, discussion de cas, trace) vit dans `algorithms/`.

N°	Tâche
4.1	Remplacer le crochet `_require_gj` par un registre générique `_procedures` (`"gauss_jordan"`, `"determinant"`, `"eigen"`…) alimenté par `algorithms/__init__.py`, sans import circulaire.
4.2	Regrouper les méthodes de commodité dans une section clairement délimitée ; vérifier qu'aucune ne dépasse quelques lignes.
4.3	Déplacer la logique de `matmul(view=…)` (vues entries/rows/columns/outer et leur trace) dans `algorithms/products.py`.
4.4	Documenter la règle dans `docs/dev/architecture.md`.

Critère de fin : `matrix.py` sous 600 lignes, API inchangée, tests verts.

9. Phase 5 — Découpage du tuteur, suppression des `eval`, API française

Cette phase est indépendante des phases 2 à 4 et peut être menée en parallèle.

9.1 Arborescence cible

Le code passe dans `tuteur/`, dont les noms deviennent l'API documentée ; `tutor/` ne contient plus que des réexportations.

```
tuteur/
  __init__.py            # API publique française (et alias anglais non documentés)
  catalogue.py           # chargement des fiches, catalogue(), themes(), résolution des références
  seance.py              # séance d'exercice, texte Markdown, fiche enseignant
  validation.py          # rapport de validation, valider_fiche, contrôle croisé
  evaluation_sure.py     # interpréteur d'expressions fondé sur ast
  verificateurs/
    __init__.py          # VerificationResult, registre type -> fonction
    commun.py            # parsing indexé, comparaisons exactes, formatage
    produit_scalaire.py  matrice.py  formule.py  valeur.py  propriete.py
    contre_exemple.py  egalites.py  dimensions.py
  indices.py  journal.py  labo.py  verrou.py  panneau.py
tutor/                   # réexportations des noms anglais, conservées au moins une version
```

9.2 Tâches

N°	Tâche	Détail
5.0	Tests préalables	Compléter `verrou` (déverrouillage : `ex.data.x.rms()` après une réponse juste), `indices` (délais et nombre minimal de tentatives) et `panneau` (widgets simulés) avant de déplacer le code (couverture actuelle : 66 %, 76 %, 24 %).
5.1	Registre des vérificateurs	Remplacer la cascade `if v_type in (...)` de la séance par un dictionnaire `{type: fonction}` ; un nouveau type = un module + une ligne d'enregistrement.
5.2	Découper `verifiers.py`	Un module par vérificateur ; `_EXPECTED_FORMS` rejoint chaque module (attribut `forme_attendue`).
5.3	Découper `catalog.py`	`seance.py` et `validation.py` ; le contrôle croisé devient une méthode par vérificateur (`controle_croise(spec, donnees)`), ce qui supprime la seconde cascade de `if`.
5.4	`evaluation_sure.py`	Interpréteur sur `ast` acceptant : nombres, noms autorisés, opérateurs arithmétiques, comparaisons, appels à une liste blanche (`Vector`, `Matrix`, `sqrt`, `Rational`, méthodes `.dot`, `.norm`…), listes, tuples, dictionnaires. Remplace les **douze** `eval`. Ne pas s'appuyer sur `sympy.parse_expr`, qui appelle `eval` en interne.
5.5	Spécification des jumeaux	Le champ `jumeau` des fiches (`generateurs.moyenne_glissante(fenetre=2, n=5)`) est analysé par l'interpréteur avec le seul espace de noms des générateurs.
5.6	API française principale	Déplacer le code de `tutor/` vers `tuteur/` ; `tutor/` réexporte les noms anglais. La documentation, le tutoriel et les carnets n'utilisent plus que les noms français. Test d'import de tous les alias anglais.
5.7	Tests	Tests dédiés à l'interpréteur (expressions refusées : attributs privés, `__import__`, lambda, compréhensions, appels hors liste blanche).

Critère de fin : aucun fichier du tuteur au-delà d'environ 350 lignes, plus aucun `eval`, diagnostics identiques, documentation entièrement en noms français. Avec les phases 3 et 4 : version **3.1.1**, **première publication sur PyPI** (l'API du tuteur est alors fixée).

10. Phase 6 — Valeurs propres, espaces propres, diagonalisation

10.1 Fonctionnalités

Élément	Description
`CharacteristicPolynomial(A)`	$\chi_A(\lambda) = \det(A - \lambda I)$ calculé **par Bareiss**, avec λ déclaré comme indéterminée du moteur (jamais discutée par l'oracle) ; résultat factorisé sur $\mathbb{Q}$ (et sur $\mathbb{Q}$(paramètres)). La convention $\det(A - \lambda I)$ est rappelée dans la trace (elle diffère de $\det(\lambda I - A)$ d'un facteur $(-1)^n$).
`Eigen(A, *, assumptions=None)`	Valeurs propres réelles avec multiplicités algébriques ; pour chacune, base de l'espace propre $\ker(A - \lambda I)$ via le moteur, multiplicité géométrique.
`EigenResult`	`char_poly`, `eigenvalues` (dict valeur → multiplicité), `eigenspaces` (dict valeur → tuple de vecteurs), `diagonalizable`, `P`, `D`, `non_real_part` (facteur sans racine réelle), `trace`.
`Matrix.eigenvalues()`, `Matrix.eigenvectors()`, `Matrix.diagonalize()`, `Matrix.char_poly()`	Méthodes de commodité.
Puissances	`Matrix.power(n)` : par diagonalisation si possible (PD^nP^{-1}, avec `n` symbolique), sinon par décomposition $\lambda I + N$ lorsque N est nilpotente (comme l'exercice 20 du cahier), sinon produits successifs pour `n` entier.

10.2 Choix mathématiques

1. **Corps de travail (décidé)** : $\mathbb{R}$ seul en 3.2. On ne renvoie que les valeurs propres **réelles** et on signale explicitement la partie du polynôme sans racine réelle (« $\lambda^2 + 1$ n'a pas de racine réelle : A n'est pas diagonalisable sur $\mathbb{R}$ »). Une option `field="C"` pourra venir plus tard.
2. **Racines non explicites** : au-delà du degré 4, ou pour des polynômes irréductibles de degré 3–4 dont les formules sont illisibles, SymPy produit des `CRootOf`. On accepte les racines rationnelles et quadratiques explicites ; pour les autres, on renvoie le facteur irréductible avec un message pédagogique, sans formules de Cardan.
3. **Paramètres** : la diagonalisabilité dépend souvent d'un paramètre (valeurs propres qui coïncident). Les valeurs critiques s'obtiennent par le discriminant ou le résultant des facteurs de χ_A ; elles donnent les branches du `CaseSplit`, dans lesquelles le moteur calcule les noyaux $\ker(A - \lambda I)$.

10.3 Pédagogie

- Trace en trois temps : polynôme caractéristique (avec la trace de Bareiss), racines (factorisation expliquée), espaces propres (élimination tracée sur $A - \lambda I$).
- Diagnostics clairs : « multiplicité géométrique 1 < multiplicité algébrique 2 pour $\lambda = 3$: non diagonalisable ».
- Vérification : $AP = PD$ contrôlé et tracé.
- Tuteur : thème « Valeurs propres et diagonalisation », 5 fiches (polynôme caractéristique 2×2, espace propre, diagonalisabilité selon un paramètre, A^n par diagonalisation, chaîne de Markov à deux états). Nouveaux vérificateurs `couple_propre` (vérifie $Av = \lambda v, v \neq 0$) et `diagonalisation` (vérifie P inversible et $P^{-1}AP$ diagonale).
- Problèmes d'exploration débloqués : dynamique de populations (Leslie), chaînes de Markov à long terme, suites récurrentes linéaires (Fibonacci).

10.4 Tests

- Comparaison avec `sympy.Matrix.eigenvects()` sur des matrices à valeurs propres rationnelles.
- Matrices de Jordan non diagonalisables, matrices symétriques (toujours diagonalisables), rotations (pas de valeur propre réelle).

- Matrices paramétrées : $\begin{bmatrix} 1 & a \\ 0 & 1 \end{bmatrix}$ (diagonalisable si et seulement si $a = 0$) ; matrice dont deux valeurs propres coïncident pour une valeur du paramètre.
- Polynôme caractéristique : aucune discussion de cas sur λ dans la trace.
- A^n symbolique vérifié pour $n = 0, \dots, 8$.

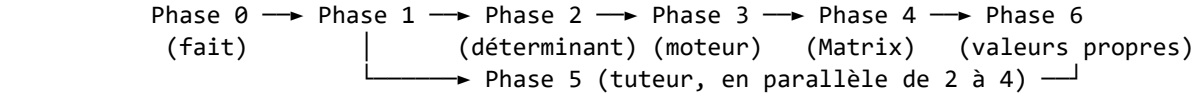
Critère de fin : `Eigen` et `diagonalize()` disponibles, 5 fiches, documentation — version **3.2.0**.

11. Phase 7 — Extensions ultérieures

Classées par rapport intérêt / effort décroissant.

Piste	Description	Dépend de
Exercices paramétrés par graine	Données d'une fiche générées à partir d'une graine (numéro d'étudiant) ; les vérificateurs, qui contrôlent des propriétés, restent inchangés. Levier fort contre la copie.	phase 5
Sous-espaces comme objets	Classe `Subspace` : engendré, noyau, image ; dimension, base, somme, intersection, supplémentaire orthogonal, projection.	phase 3
Géométrie de $\mathbb{R}^2$ et $\mathbb{R}^3$	Produit vectoriel, droites et plans, intersections, distances, positions relatives avec discussion de cas, figures 3D.	phases 2 et 3
Factorisation LU (avec permutation)	Tracée comme Gauss-Jordan, reliée aux substitutions triangulaires.	phase 3
QR par Householder / Givens	Pour enrichir les comparaisons numériques.	—
Rejeu de trace dans JupyterLab	Curseur « étape suivante » sur les traces, pivot et lignes mis en évidence.	—
Générateur de sujets LaTeX/PDF	Feuilles d'exercices et sujets à variantes, avec corrigés.	exercices paramétrés
Parcours adaptatif et tableau de bord enseignant	Graphe de prérequis, agrégation des journaux d'une classe.	phase 5
Locale anglaise	Fichier `en.yaml`.	—
Documentation en ligne	Site HTML publié (GitHub Pages) à chaque version.	phase 1

12. Calendrier et dépendances



Ordre	Phase	Séances	Jalon	Version
—	Phase 0	fait	tests de caractérisation en place	—
1	Phase 1	1–2	corrections, dépôt public propre, CI verte sur GitHub	3.0.1
2	Phase 2	2–3	`det()` disponible (Bareiss par défaut avec paramètres), 4 fiches	3.1.0
3	Phase 3	2–3	`GaussJordan` et `Determinant` sur le moteur, sorties identiques	—
4	Phase 4	1	`Matrix` < 600 lignes, registre de procédures	—
5	Phase 5	2	tuteur découpé, plus d'`eval`, API française	3.1.1 + PyPI
6	Phase 6	3–4	`Eigen` et `diagonalize()`, 5 fiches	3.2.0

Total indicatif : **11 à 15 séances** jusqu'à la version 3.2.0.

13. Risques et parades

Risque	Probabilité	Impact	Parade
Régression invisible pendant les refactorisations	moyenne	fort	Tests de caractérisation, commits atomiques, comparaison des carnets exécutés avant / après.
Moteur conçu trop tôt pour un seul usage	moyenne	moyen	Déterminant livré avant le moteur ; besoins de la phase 6 (indéterminées) pris en compte dès la conception.
Trace fausse pour certaines valeurs des paramètres	moyenne	fort	Bareiss par défaut avec paramètres ; correction de l'oracle (tâche 1.1) ; principe « une trace valable sur toute sa branche ».
Explosion de la taille des expressions (paramètres, 4×4 et plus)	forte	moyen	Bareiss (taille des entrées maîtrisée), factorisation systématique, limite de taille documentée, message explicite au-delà.
Racines non explicites (degré ≥ 3)	forte	moyen	Politique claire (§ 10.2), facteurs irréductibles renvoyés tels quels.
Contenu protégé publié par erreur	faible	fort	Retrait du PDF de l'historique **avant** le premier `push` ; attribution des sources (tâches 1.6, 1.7).
Réécriture d'historique après publication	faible	moyen	`git filter-repo` uniquement avant le premier `push` ; ensuite, plus aucune réécriture des branches publiées.
Import circulaire lors du registre de procédures	faible	moyen	Registre alimenté par `algorithms/__init__.py`, comme le crochet actuel.
Synchronisation OneDrive gênant git ou l'écriture de fichiers	moyenne	faible	Réessayer les écritures ; si possible, travailler hors du dossier synchronisé.

14. Décisions

14.1 Décisions prises (20 octobre 2026)

1. `requires-python >= 3.10`.
2. Valeurs propres sur $\mathbb{R}$ seul en 3.2 ; $\mathbb{C}$ éventuellement plus tard.
3. Déterminant : élimination, cofacteurs, Sarrus **et Bareiss**, Bareiss étant la méthode par défaut quand la matrice contient des paramètres.
4. Ordre : déterminant avant extraction du moteur.
5. Publication : dépôt GitHub public, puis PyPI à partir de 3.1.1.
6. API du tuteur : le français est l'API documentée ; les noms anglais (`tutor`, `verify_*`…) restent des alias non documentés pendant au moins une version.
7. Versions intermédiaires : 3.0.1 (corrections et publication), 3.1.0 (déterminant), 3.1.1 (refactorisations, PyPI), 3.2.0 (valeurs propres).

14.2 Questions encore ouvertes

1. **Langue du reste de l'API** : `Matrix.rref()`, `explain()`, `GaussJordan`… restent en anglais (avec des messages en français) ; faut-il des alias français pour le cœur aussi, ou seulement pour le tuteur ?
2. **Carnets d'essai** : `essai1` à `essai4` — à supprimer, ou à conserver hors du dépôt public ?
3. **Fiches adaptées de VMLS** : selon les conditions du recueil, faut-il simplement attribuer, demander l'accord des auteurs, ou reformuler davantage certaines fiches ?
4. **Alias anglais** : durée de conservation au-delà d'une version, et faut-il émettre un `DeprecationWarning` ?

15. Définition de « terminé » (pour chaque étape)

- ☐ Tests unitaires et de caractérisation verts (`pytest`), carnets exécutés sans erreur (`python verifier_tout.py`).
- ☐ `validate_sheet` / `valider_fiche` valide les 35 fiches (et les nouvelles).
- ☐ `ruff` sans erreur ; `mypy` sans nouvelle erreur (et sans erreur du tout sur les dossiers où il est bloquant).
- ☐ Toute référence régénérée a été relue (diff commenté dans le message de commit).
- ☐ Documentation Markdown mise à jour, site HTML régénéré (`python build_docs_site.py`), liens vérifiés.
- ☐ `CHANGELOG.md` complété.
- ☐ Commit sur `development` avec un message explicite ; CI verte sur GitHub.

Annexe A — Point de reprise pour la prochaine séance

1. Commiter ou annuler les sorties des cinq carnets modifiés ; trier `essai3` et `essai4`.
2. Tâche 1.1 : relever les valeurs interdites des dénominateurs dans l'oracle (`core/zero.py`), écrire les tests, régénérer et relire les références.
3. Tâches 1.2 à 1.4 : un commit chacune.
4. Tâche 1.6 : retirer le manuel PDF de l'historique (`git filter-repo`), **avant** de créer le remote GitHub (tâche 1.9).

Annexe B — Esquisse de l'API cible (version 3.2)

```
from linalg_pedagopy import Matrix, symbols, Determinant, Eigen

a, = symbols("a")
A = Matrix([[a, 1, 1], [1, a, 1], [1, 1, a]])

A.det()                                    # (a - 1)**2*(a + 2), calculé par Bareiss
Determinant(A).run().zeros()               # a = 1, a = -2
Determinant(A, method="cofactors").run().explain(level=2)
Determinant(A, method="elimination").run()
# 3.1.0 : arrêt expliqué (pivot a – 1 conditionnel) ; à partir de la phase 3 : CaseSplit

B = Matrix([[2, 1], [1, 2]])
res = Eigen(B).run()
res.eigenvalues                            # {1: 1, 3: 1}
res.eigenspaces                            # {1: (Vector([-1, 1]),), 3: (Vector([1, 1]),)}
P, D = B.diagonalize()
n, = symbols("n")
B.power(n)                                 # P D**n P**-1, en fonction de n

Matrix([[1, a], [0, 1]]).diagonalize()
# CaseSplit : a = 0 -> diagonalisable ; a ≠ 0 -> non diagonalisable (λ = 1 double, espace propre de dimens

from linalg_pedagopy import tuteur         # API française documentée
ex = tuteur.exercice("pythagore")
```